Expectigammon: A Backgammon Agent Using Expectiminimax Search

Josh Longo and Kevin Shi

Abstract

Backgammon poses a complex challenge for game-playing AI due to its state space exceeding 10^{20} possible board positions, high branching factor, and inherent randomness from dice rolls. This paper presents the design and implementation of a backgammon agent using an Expectiminimax search algorithm with a custom heuristic evaluation function. The board is represented as a 28-element array encoding the 24 points, bar, and bearing-off positions for both players. The search algorithm handles MAX, MIN, and CHANCE nodes by averaging over all 21 distinct dice outcomes at each CHANCE node to accurately reflect probabilistic transitions. The heuristic evaluation function includes pip count, blot penalties scaled by both board position and nearby opponent threat, consecutive points covered on a player's home board, a bonus for pieces in the homeboard, a pip-disparity penalty for pieces stranded far back in opponent territory, and borne-off piece count. To manage the large branching factor, alpha-beta pruning is applied at MAX and MIN nodes, and forward pruning limits the collection of moves at each node. Informed by Tesauro's work on TD-Gammon, which demonstrates that strong evaluation quality can compensate for limited search depth [1], we prioritized heuristic design over deep search. The agent is tested through simulated self-play, and the results show that the heuristic and pruning together produce skillful, strategically reasonable play within time limits to allow for human play. We discuss the strengths and weaknesses of a handcrafted heuristic compared to learned evaluation functions, and propose directions for future improvement.

1. Introduction

Backgammon is a two-player board game where each player moves 15 checkers in opposite directions around the board, attempting to be the first to move all their pieces off the board. Although artificial intelligence has made considerable progress in board games, backgammon presents a tough challenge because it involves dice rolls that determine piece movement. This randomness dramatically increases the branching factor of the search space, making it difficult to find the optimal move. The full game contains an estimated 10^{20} possible board states, making an exact solution infeasible [1]. Each turn involves 21 possible dice outcomes, with an average of about 20 legal moves per outcome, resulting in hundreds of potential branches per ply [1]. Compared to chess, where branching factors are typically between 30 and 40, backgammon's search space grows too rapidly to permit deep minimax search [1], stressing the need for robust positional evaluation rather than relying on deep search.

Before TD-Gammon, traditional backgammon programs primarily relied on handcrafted heuristic evaluation functions developed in collaboration with human backgammon experts, encoding features such as blockade strength, race position, and hitting potential. Tesauro notes several problems with this approach, including the large number of features and the difficulty of articulating and coding the relevant ones, the potential for nonlinear relationships between features that are not properly weighted, and the possibility of bias or outdated strategic assumptions introduced by human experts [1]. These limitations bring up concerns about relying on a manually engineered heuristic rather than allowing a system to learn its own evaluation function. In his groundbreaking paper, Programming Backgammon Using Self-Teaching Neural Nets, Gerald Tesauro introduced TD-Gammon, a reinforcement learning system that achieved world-class, and eventually superhuman, performance through self-play and neural network evaluation [1]. TD-Gammon addresses these limitations using a multi-layer perceptron neural network trained through temporal difference learning, observing sequences of board states from self-play and updating its evaluation estimates based on prediction errors between successive states [1].

A relevant insight from Tesauro's work is that evaluation quality can compensate for limited search depth. Early versions of TD-Gammon only evaluated positions resulting from immediate moves, without simulating deeper opponent responses, while subsequent versions incorporated shallow 2-ply and 3-ply searches that employed probabilistic averaging over dice outcomes rather than full-width minimax backup [1]. This hybrid approach of strong evaluation with shallow lookahead demonstrates that in nondeterministic and stochastic games with large branching factors, improving evaluation accuracy may yield greater gains than increasing search depth. Additionally, because backgammon involves chance nodes representing dice rolls, decisions must be based on expected values across possible outcomes rather than assuming a single deterministic continuation, and TD-Gammon incorporated this expectation step during shallow search while limiting expansion through pruning to prevent the tree from growing uncontrollably [1].

Like TD-Gammon, our agent must operate under strict depth limitations and a high branching factor caused by stochastic dice outcomes, meaning evaluation quality plays a central role in performance. However, Tesauro also points out that features often interact in complex and unanticipated ways, and there is no principled method for assigning correct weights to those interactions [1], meaning even reasonable features such as race position or blockade strength may be nonlinear and difficult to capture with simple weighting. Informed by these conclusions, this project presents the design and implementation of a search-based backgammon agent using Expectiminimax search with a custom heuristic evaluation function, directly addressing these weaknesses through iterative heuristic development and systematic evaluation via simulated self-play.

2. Methods

This section explains the design and implementation of the Expectigammon agent, describing the board representation, search algorithm, heuristic evaluation function, pruning strategies, and evaluation methodology.

2.1 Board Representation

The board is represented as a 28-element array, with the starting position illustrated in **Fig. 1** below. Indices 0 through 23 correspond to the 24 points of the board, with index 0 representing point 1 and index 23 representing point 24, ascending in order.

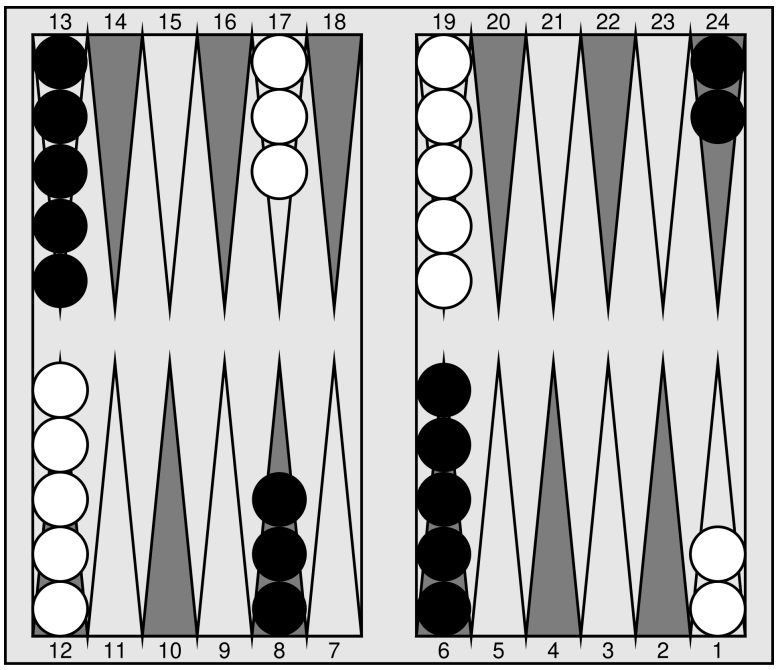

Fig. 1. Checkers in starting position, from [2]

White pieces are stored as positive integers and black pieces as negative integers at each index, allowing move validation to be determined directly from the sign and count of each element. For instance, in the starting position shown in Figure 1, index 0 holds a value of 2 representing two white pieces on point 1, and index 5 holds a value of -5 representing five black pieces on point 6. This sign convention makes move validation straightforward, white can never move to a point with a value less than -1, and black can never move to a point with a value greater than 1, a condition that is directly readable from the array value at that index. The remaining four indices handle the game's special positions. Indices 24 and 25 store the number of white and black pieces on the bar, respectively, and indices 26 and 27 track the number of pieces each player has successfully borne off. When either reaches 15, it indicates a win. All

four of these values are stored as positive integers since they represent counts rather than directional positions. Legal move generation handles all standard backgammon rules, including bar re-entry, bearing off, and doubles as four moves rather than two, generating all legal moves and passing them to the search algorithm as candidate actions.

2.2 Expectiminimax Algorithm

The agent uses an Expectiminimax search algorithm that builds upon the classical Minimax framework to handle the random nature of backgammon. Each node in the search tree represents a specific board state, and the node type determines what happens next from that state. MAX nodes represent board states where the agent has a known roll and selects the move sequence that maximizes the value returned by the subtree beneath them, the expected value propagated up from the CHANCE node below when the search continues deeper, or the heuristic score returned by the evaluation function when the depth limit has been reached. MIN nodes represent board states where the opponent has a known roll and analogously select the move sequence that minimizes the value returned by the subtree beneath them under the same conditions. CHANCE nodes represent board states where the next roll is unknown and branch into all 21 distinct dice outcomes before reaching a player decision node. Because dice rolls are independent and memoryless in backgammon, every CHANCE node, regardless of its position in the tree, always considers the full set of 21 outcomes; a previous roll has no influence on future rolls. Non-double outcomes are weighted at 2/36 and doubles at 1/36, showing that two orderings are the same for a non-double roll versus the single ordering of doubles. The expected value computed at a CHANCE node is therefore a probability-weighted sum of the optimal values returned beneath it. Each of the 21 player decision nodes below first selects the optimal move for that specific roll, and those optimal values are then weighted and summed into a single expected value that propagates back up the tree. The full tree pattern is illustrated in **Fig. 2** below.

Fig. 2. Alternating MAX, MIN, and chance nodes in the expectiminimax tree.

Although the tree alternates between MAX and MIN nodes, as shown in **Fig. 2**, both players always maximize their own score. The heuristic score is normalized to the current player's perspective before being used in the search, so what appears externally as a minimizing operation is, from the opponent's perspective, an internally maximizing operation. Rather than implementing CHANCE nodes as a separate recursive step, the probability averaging over all 21 dice outcomes was incorporated directly into the MAX and MIN node logic, with each player decision node looping over all 21 outcomes internally, computing the optimal move for each, and averaging the results into a single expected value. The root of the search tree is handled separately from the recursive architecture. At the start of each turn, the agent rolls the dice and evaluates all possible move sequences for the known roll before invoking the recursive search on the resulting board states, meaning the recursive function handles only the subtree beneath the agent's first move rather than the full tree from the root. Depth decrements only at player decision nodes rather than at CHANCE nodes, since CHANCE nodes perform no strategic decision and serve purely as probability averaging layers between player turns. To avoid redundancy when different move sequences converge on the same board state, a transposition table stores previously evaluated board positions along with the remaining search depth and whose turn it is, returning the cached expected value immediately if the same position has already been evaluated under identical conditions. When no legal moves exist for a given roll, the turn is skipped, and the search continues from the current board state, making sure the tree handles forced passes without prematurely terminating. The search terminates when the specified depth is reached, or a terminal game state is detected, at which point the heuristic evaluation function is applied to score the resulting board position.

2.3 Heuristic Evaluation

Our heuristic function produces an evaluation of the current board state, with positive values denoting an advantage for white and negative values reflecting an advantage for black. This is the move score displayed in the graphical interface and in the terminal during game simulation. Within the search, however, this score is flipped to the current player’s perspective, meaning both players always act as the MAX node trying to maximize a positive value relative to themselves, rather than one player maximizing and the other minimizing an absolute score. The function evolved over three iterations, with the initial one evaluating board positions using a single feature: pip count, which measures the total distance each player's pieces must travel to bear off. For each piece on the board, we calculated the remaining distance to the bearing-off area and subtracted it from the heuristic score, rewarding positions closer to the bearing-off area and penalizing those farther away. While pip count is the most fundamental measure of progress in backgammon and provides a reasonable baseline for decision-making, it fails to fully capture

critical tactics, such as the risk of exposed blots, the severity of having pieces on the bar, and the advantage of building blockades on the home board.

To handle the restrictions of the initial heuristic, we introduced five additional features: a blot penalty to discourage leaving single exposed pieces, a bar penalty to heavily penalize pieces sent to the bar, a prime bonus to reward building consecutive blocked points, a home board coverage bonus to encourage clustering pieces in the home board, and a bearoff bonus to reward pieces that had successfully borne off. These all reflect the advantage or disadvantage that having pieces in these positions creates, with the blot penalty also applying a higher danger multiplier to blots in the opponent's home board, where they were most vulnerable to being hit.

After observing gameplay from the second version of our heuristic, we identified two remaining weaknesses: the agent was not aggressive enough about penalizing pieces stranded deep in the opponent's home board as the game developed, and the prime bonus was rewarding consecutive blocked points anywhere on the board rather than specifically in the home board, where primes are strategically meaningful. In the final version we restricted the prime bonus to the home board only, redesigned the blot penalty to scale with the number of nearby opponent pieces that could hit the blot, introduced a symmetric stuck penalty for both players' pieces stranded deep in opponent territory, and made the stuck penalty weight responsive to pip count disparity between the two players to encourage escaping back pieces during the race phase.

2.4 Pruning Strategies

The Expectiminimax algorithm implements two pruning strategies to diminish computational cost and improve the speed of each move. The first strategy is forward pruning combined with move ordering. Rather than evaluating every possible move sequence for a given roll, the algorithm first scores each possible moveset by applying it to a copy of the board and evaluating the resulting position with the heuristic function. These scores are then used to sort the movesets in descending order for MAX nodes and ascending order for MIN nodes, making sure the most promising moves are evaluated first. The sorted movesets are then capped at a fixed number, discarding the remaining moves. Move ordering is applied only at the root node, where the ranking quality justifies the additional computational cost of scoring every prospective move sequence. In recursive calls, the moveset is capped early during generation without ordering, since applying full move ordering at every level of the tree would be far too expensive given the branching factor. Move ordering is essential to making forward pruning meaningful at the root because, without first ranking moves by heuristic score, capping the candidate set would discard moves arbitrarily rather than selectively retaining the most promising ones.

Secondly, alpha-beta pruning was applied at each player's decision node to eliminate paths that cannot influence the final decision. Alpha-beta pruning maintains a lower bound alpha and upper bound beta, pruning branches where the current best value for a MAX node exceeds beta or the current best value for a MIN node falls below alpha, since the opposing player would never allow play to reach those states. Because CHANCE nodes are folded directly into the MAX and MIN node logic in this implementation, alpha-beta bounds are reset independently for each of the 21 roll outcomes within a single player node rather than being carried across the entire CHANCE computation. Together, these two strategies significantly reduce the number of nodes evaluated at each turn, allowing the agent to search at a practical depth within reasonable time constraints.

2.5 Evaluation

The agent was tested through simulated self-play, with it competing against itself over a series of games. We compared our heuristic functions across 50 games with a depth of 1 and a move cap of 5, so we could distinguish the effects of the heuristics while still running a large number of simulations. We then compared depth by running simulations again, this time over only 10 games per configuration, using the final heuristic at depths 1 and 2. Because the computational cost of depth 2 search made larger sample sizes impractical, the time constraints did not justify the extra simulations. Our hyperparameters were not increased beyond a depth of 2 and a move cap of 5, given backgammon's large branching factor, which makes deeper searches computationally burdensome and can lead to turns taking minutes, if not hours. Performance was assessed by using metrics tracked throughout each game, including the win rate between the two players, the total number of turns per game, and the average game time for heuristic comparison. Nodes visited and pruned per game were additionally tracked for depth comparison. Lastly, a turn limit of 80 was imposed to prevent games from running indefinitely when neither player could achieve a decisive advantage.

3. Results

This section discusses the results of our agent’s simulated self-play experiments, evaluating the agent across three heuristic versions, two search depths, and a demonstration of the graphical interface.

3.1 Heuristic Comparison

The first heuristic, which evaluated board positions solely using pip count along with basic blot and bar penalties, achieved a Player 1 win rate of 78%, with an average of 45.1 turns per game and a game time of 3.8 seconds. Many games included legitimate skipped turns due to pieces being unable to re-enter from the bar, reflecting the closed-board scenarios that backgammon strategy naturally produces. No games reached the 80-turn limit, which showed that the pip count heuristic, despite its simplicity, was

sufficient to drive games to a decisive conclusion. The relatively high win rate for player 1 against identical agents is consistent with the inherent advantage of being first to move in backgammon, which decreases pip count. This was not a problem with the evaluation function, but more a baseline advantage that had to be kept in mind when interpreting win rates across all heuristic comparisons.

The second heuristic earned an 80% win rate, with an average of 44.9 turns per game and an average game time of 3.5 seconds. While the win rate improvement over the first heuristic was marginal, the most significant difference was the large increase in skipping turns due to pieces on the bar, with multiple games featuring extended sequences in which a player was unable to re-enter from the bar. This behavior is directly caused by the home board coverage bonus and the strengthened blot and bar penalties introduced in this version, which incentivize the agent to build closed home boards and aggressively send opponent pieces to the bar. This pattern reflects the more positionally aware play that these features were designed to enable. Games also completed slightly faster on average, suggesting that the additional strategic features helped drive positions toward decisive conclusions more efficiently than the pip-count baseline alone.

The final heuristic, using the same evaluation, had a win rate of 68%, an average of 46.2 turns per game, and an average game time of 3.5 seconds. The win rate decreased from 80% to 68% in the final heuristic, showing more balanced and competitive play between the two agents. Additionally, it proved capable of subduing and reducing the first-mover advantage that had dominated earlier heuristic versions. There was also an improvement in how the heuristic led to even more dynamic positional play, with games featuring more frequent bar-trapping sequences and blots left in strategically acceptable positions, such as pieces with no nearby opponent threats, rather than avoided entirely out of an overly conservative evaluation. The pip disparity stuck penalty also produced an obvious behavioral change in the race phase, with back pieces escaping opponent territory earlier rather than remaining anchored deep in the opponent's home board as the game transitioned toward a pure race. This naturally introduced more contested and unpredictable positions that occasionally benefited the second player. Games were also slightly longer on average at 46.2 turns compared to 44.9 for heuristic 2, consistent with the more contested positional play the updated evaluation function encourages.

3.2 Depth Comparison

Looking at **Table 1** below, we can see that increasing search depth from 1 to 2 produced meaningful differences across metrics. The win rate slightly increases from 40% to 60% in favor of player 1, implying that a deeper search provides a larger advantage, allowing the agent to anticipate the opponent's responses by one full round. The lack of pruning at depth 1 is expected, since alpha-beta pruning requires both a MAX and a MIN node to compare values against, and at depth 1, the search ends

after a single player decision level, leaving no opportunity for pruning. At depth 2, however, approximately 58% of nodes were eliminated through pruning, demonstrating that alpha-beta combined with move ordering helps to manage the branching factor. The most significant difference is computational cost, shown by average game time increasing from 3.9 seconds to 296.8 seconds, around 75 times more, while nodes visited grew from about 1,319 to 85,084. This confirms that although a deeper search improves play quality, the exponential growth of the search tree makes depth 2 a good limit where the agent still plays successfully and competitively. As Tesauro mentioned in the TD-Gammon paper, shallow search paired with a strong evaluation function can produce competitive play, as we saw with the depth-1 agent, which played quite strategically despite its limited lookahead. The depth-2 configuration with a move cap of 5 represents the final agent configuration used in the graphical user interface. This shows that a strong, targeted heuristic function can compensate for the lack of depth in the search tree.

Table 1: Depth Comparison Statistics Over 10 Games"

Configuration	Player 1 Win Rate	Average Turns	Average Time (Seconds)	Average Nodes Visited	Average Nodes Pruned
Depth = 1 Moves Cap = 5	40.00%	43.4	3.89	1318.70	0.00
Depth = 2 Moves Cap = 5	60.00%	46.4	296.75	85082.60	49613.70

3.3 Graphical Interface

Fig. 3 below displays the graphical interface developed to support both Human vs AI and AI vs AI gameplay. The right panel displays the current game state, including the dice roll, borne off piece counts, bar counts, the AI agent's most recently executed move sequence, and the expected value of that move. The expected value of -48.00 means a strong positional advantage for Black following the AI's move, which makes sense since Black has successfully advanced pieces toward the home board while White retains back pieces at point 12. Expected values are displayed in the absolute convention established in the heuristic section. Positive values indicate an advantage for White and negative values indicate an advantage for Black, allowing both players to assess the current game state at a glance.

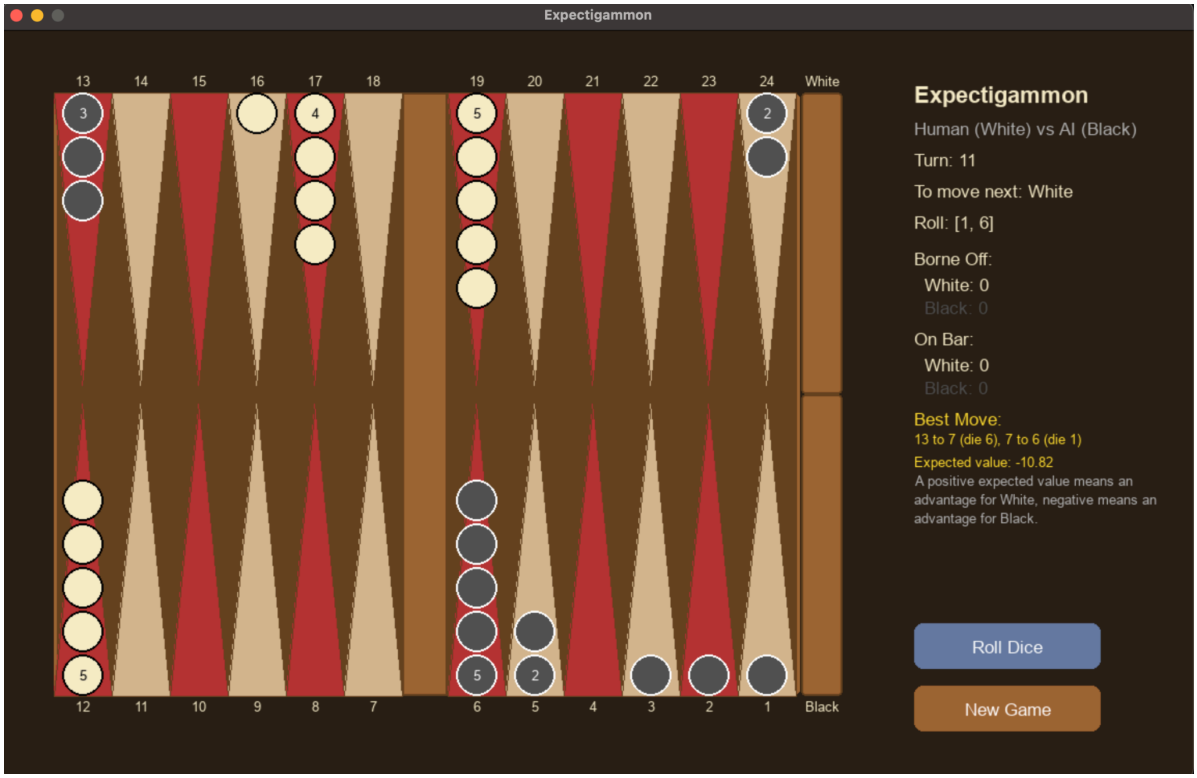

Fig. 3. Sample mid-game board state from a Human vs AI game at turn 11, displaying the agent's recommended move sequence and expected value computed by the Expectiminimax search**.**

4. Conclusions

Overall, we created a fairly effective backgammon agent, though somewhat limited in playing ability. The results support Tesauro's main conclusion that evaluation quality can compensate for limited search depth [1]. Our depth-1 agent has already shown calculated play, including home board development, bar trapping, and race awareness. However, depth 2 showed tangible improvement in win rate at the cost of a roughly 75 times increase in computational cost. By avoiding the storage of play patterns and using careful heuristics and pruning, we limited memory usage and found a powerful balance between performance and decision time. This confirmed that, in stochastic games with large branching factors, investing in a stronger evaluation function can lead to greater success than forcing deeper search [1].

Additionally, our results also verify Tesauro's warning about the limitations of handcrafted heuristics. Features often interact in complex and unanticipated ways, and it is extremely difficult to assign appropriate weights to these nonlinear relationships [1]. This was evident in our iterative heuristic development, as adjusting the blot penalty, prime bonus, and stuck penalty required repeated observation of gameplay rather than a systematic method to optimize those numbers. The agent's tendency to keep

pieces anchored deep in the opponent's home board and prioritize moving pieces farther up into their homeboard even as the game transitioned into the race phase demonstrated this. This is exactly the subtle positional relationship that simple weighted features find it hard to capture.

Comparatively, TD-Gammon replaced handcrafted features entirely with a neural network trained through temporal difference learning [1], while our agent relied on a fixed evaluation function. This method makes our model vastly more interpretable, as each feature in our heuristic has a logical justification grounded in understanding gameplay and the agent's behavior, and a value that can be traced back through trial-and-error simulations. However, TD-Gammon's learned evaluation naturally captures nonlinear feature interactions through its network weights [1]. Our linear weighted sum cannot represent the complicated relationships among features such as prime strength, race position, and blot danger that expert play requires, and so performs understandably worse.

For future improvement, we could integrate a self-play reinforcement learning component to fine-tune or replace the evaluation function. Rather than manually tuning weights, a temporal difference learning approach could observe sequences of board states from self-play and update evaluation estimates based on prediction errors [1]. This would allow the agent to discover feature interactions that are difficult to articulate explicitly. In addition, we could implement further pruning strategies to lower the computational cost of deeper search, making depth 3 or beyond realistic on standard computers. This would similarly align it more closely to the ideal blended approach of strong evaluation combined with shallow lookahead that made TD-Gammon such a groundbreaking and successful Backgammon agent [1].

References

[1] G. Tesauro, "Programming backgammon using self-teaching neural nets," *Artificial intelligence*, vol. 134, no. 1, pp. 181–199, 2002, doi: 10.1016/S0004-3702(01)00110-2

[2] B. Robertie, *Backgammon for Winners*. New York: Cardoza Publishing, 1995.